

G17 RESEARCH PROJECT

Training notebook

A detailed walkthrough of paper-based
AMFU-Net learning and evaluation

The notebook moves from dataset shards to a published model

The cells follow the same order needed for a repeatable experiment.

Load

Authenticate, open the Parquet splits and create PyTorch datasets.

Build

Define AMFU-Net, DAG, MFEF, loss and metrics.

Train

Run the paper settings, resume checkpoints and monitor validation.

Evaluate

Test once, save evidence and push the model to Hugging Face.

The dataset loads directly from Hugging Face shards

The notebook avoids downloading thousands of individual PNG files.

```
dataset = load_dataset(  
    "aee4/G17-preprocessed-dataset"  
)  
train_data = dataset["train"]  
val_data   = dataset["validation"]  
test_data  = dataset["test"]
```

What it means

Hugging Face Datasets reads compact Parquet/Arrow files. Each row contains the enhanced image, mask and metadata. The three predefined splits remain unchanged.

SegmentationDataset creates the tensors used by PyTorch

Every row becomes an input image and a correct binary mask.

```
return {  
    "pixel_values": torch.from_numpy(image)  
                        .unsqueeze(0),  
    "labels": torch.from_numpy(mask)  
                .unsqueeze(0),  
}
```

What it means

pixel_values has one grayscale channel. labels is 1 where the expert mask marks lesion and 0 elsewhere. The original dataset row metadata is read again later when per-image test results are saved.

Training augmentation changes both image and mask together

Random transformations increase variety without changing the answer.

Geometric changes

Flip, rotate, shift and scale the pair using identical coordinates.

Why identical?

If only the image moved, the mask would point to the wrong location.

Training only

Validation and test images are resized and normalised but not randomly changed.

Resizing uses different interpolation for images and masks

The notebook converts every example to 256×256 pixels.

```
image = cv2.resize(image, (256,256),  
                    interpolation=cv2.INTER_AREA)  
mask = cv2.resize(mask, (256,256),  
                   interpolation=cv2.INTER_NEAREST)
```

What it means

Area interpolation is suitable when reducing grayscale images. Nearest-neighbour preserves mask values as exact classes and avoids creating uncertain grey labels at the lesion boundary.

AMFU-Net is an encoder–decoder segmentation network

Its output has the same spatial purpose as the input: one prediction for every pixel.

Encoder

Reduces image size while learning increasingly complex features.

Bottleneck

MFEF combines information from several receptive-field scales.

Decoder

Restores spatial resolution and uses encoder skip features.

Output

A one-channel logit map becomes lesion probability after sigmoid.

DAG filters each skip connection

Dual Attention Gate is applied at four decoder levels.

```
f1 = sigmoid(self.branch_average(x))
f2 = sigmoid(self.branch_maximum(x))
g1 = adaptive_avg_pool2d(f1, 1)
g2 = adaptive_max_pool2d(f2, 1)
return self.output(x * g1 + x * g2)
```

What it means

The notebook creates two channel-weight summaries using average and maximum pooling. Their gated versions of the skip feature are combined before decoding.

DAG combines average and maximum attention paths

The two branches build complementary channel gates from the skip feature.

Average path

A learned feature is summarised with global average pooling.

Maximum path

A second learned feature is summarised with global maximum pooling.

Gate output

Both channel-weighted skip tensors are added and passed through an output convolution.

MFEF studies patterns at several effective scales

The bottleneck module fuses parallel feature transformations.

```
multi_scale = torch.cat([
    self.conv1(x), self.conv3(x),
    self.conv5(x), global_feature
], dim=1)
w1 = softmax(self.compress(multi_scale), dim=1)
w2 = softmax(self.dilated(x), dim=1)
return x * (0.5*w1 + 0.5*w2)
```

What it means

The 1×1 , 3×3 and 5×5 filters plus global pooling capture different context sizes. A dilated branch adds wider context, and the two learned weights modulate the bottleneck feature.

The forward pass produces one logit per pixel

Encoder features are reused as gated skips during decoding.

```
s1 = self.enc1(x)
s2 = self.enc2(self.pool(s1))
s3 = self.enc3(self.pool(s2))
s4 = self.enc4(self.pool(s3))
x  = self.mfef(self.bridge(self.pool(s4)))
...
return self.head(x)
```

What it means

The output is not yet a black-and-white mask. It is a logit map. Sigmoid converts logits into probabilities; a threshold later turns those probabilities into a binary prediction.

The loss balances pixel classification and region overlap

The paper did not state a loss, so the notebook declares BCE + Dice explicitly.

```
bce = F.binary_cross_entropy_with_logits(
    logits, labels
)
dice_loss = 1 - soft_dice(
    torch.sigmoid(logits), labels
)
loss = bce + dice_loss
```

What it means

BCE judges individual pixels. Dice loss directly rewards overlap between predicted and true lesion regions. Declaring this adaptation prevents it from being mistaken for an undocumented paper setting.

The main optimiser settings follow the paper

Hardware-safe batching changes memory use without changing the intended effective batch.

Optimiser

Adam with learning rate 0.001 and weight decay $1e-5$.

Schedule

Stepwise learning-rate decay, matching the paper description.

Epochs

Up to 200 paper epochs; the completed run may stop at its configured endpoint.

Batching

Micro-batch 4 × accumulation 3 = effective batch size 12.

A custom Trainer enforces Adam instead of AdamW

This cell corrects the earlier mismatch with the paper optimiser.

```
self.optimizer = torch.optim.Adam(  
    self.model.parameters(),  
    lr=self.args.learning_rate,  
    weight_decay=self.args.weight_decay  
)  
self.lr_scheduler = StepLR(  
    self.optimizer,  
    step_size=steps_per_epoch * 10,  
    gamma=0.5  
)
```

What it means

Transformers normally defaults to AdamW and its own scheduler. The override uses Adam and halves the learning rate every ten epochs, matching the actual notebook.

Checkpoint resume protects long training runs

The notebook checks the model repository before starting.

Find checkpoint

Look for the last-checkpoint folder in the Hugging Face model repository.

Resume state

Restore weights, optimiser, scheduler, epoch and global step.

Why it matters

A Kaggle session can end without forcing training to restart from zero.

Metrics measure overlap and boundary quality

No single metric describes every segmentation error.

Dice

Overlap score; 1 means perfect agreement.

Jaccard / IoU

Intersection divided by union; stricter overlap measure.

HD95

95th-percentile boundary distance; lower is better.

ASD

Average boundary distance; lower is better.

Training improves, then levels off

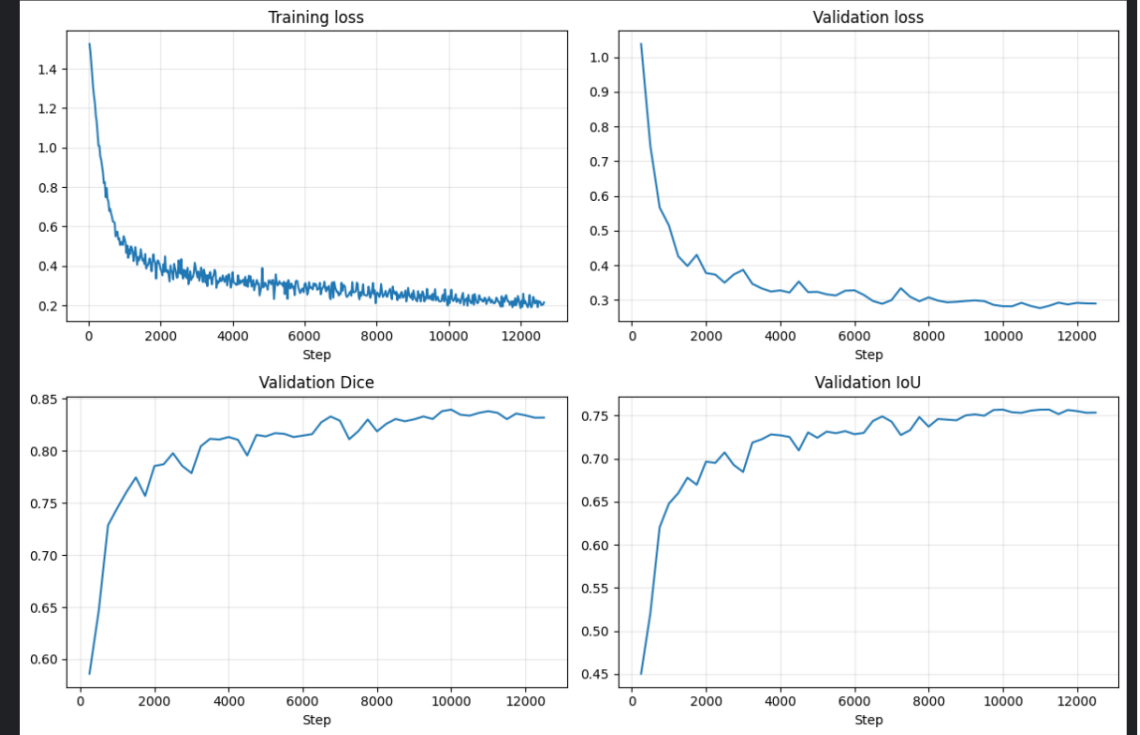
How to read the curves

Training loss falls: the model makes fewer errors.

Validation loss also falls: improvement transfers to unseen validation images.

Dice and IoU rise, then level off.

A plateau means later epochs bring smaller gains.



Internal-test evaluation runs only after training

The test split provides the final unbiased estimate inside the combined dataset.

```
test_output = trainer.predict(  
    test_set,  
    metric_key_prefix="test"  
)  
probabilities = sigmoid(  
    test_output.predictions  
)
```

What it means

The notebook reports aggregate metrics and stores per-image results. Predictions are also visualised beside the enhanced image and expert mask to reveal errors that one average number can hide.

The final cells publish a reproducible model package

Weights alone are not enough to recreate inference.

```
trainer.save_model(OUTPUT_DIR)
trainer.push_to_hub(
    commit_message="Complete G17 AMFU-Net training"
)
hf_api.upload_folder(
    folder_path=str(RESULTS_DIR),
    path_in_repo="results"
)
```

What it means

The repository receives model weights, configuration, training state, metric files, curves and example predictions. The demo can then load the same architecture and weights.

What the completed training result does—and does not prove

The model learned useful segmentation patterns, but evaluation scope still matters.

Supported

Good internal-test overlap shows the network can outline lesions in data drawn from the prepared sources.

Not yet supported

It does not prove clinical safety, diagnosis ability or reliable performance on every healthy image.

Next scientific step

Evaluate the fixed model on a genuinely external dataset without retraining.